Name:Mounashree Prasanna
SJSU ID: 0175868333

DATA 236 - Assignment 2

Part 1. HTML & CSS

```html
<html lang="en">

<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <link rel="stylesheet" href="/css/styles.css">
</head>

<body>
    <div class="container">
        <h1 class="heading">Book Store App</h1>
    </div>

    <h2>List of All Books</h2>
    <table class="books-table">
        <thead>
            <tr>
                <th>Book ID</th>
                <th>Title</th>
                <th>Author</th>
            </tr>
        </thead>
        <tbody>
        <% books.forEach(function(book){ %>
            <tr>
            <td><%= book.BookID %></td>
            <td><%= book.Title %></td>
            <td><%= book.Author %></td>
            <td>
                <a href="/update-book/<%= book.BookID %>">
                <button>Update</button>
                </a>
            </td>
            </tr>
        <% }); %>
        </tbody>
    </table>

    <!-- Update User -->
    <div class="actions-container">
        <a href="/add-book">
            <button>Add Book</button>
        </a>

        <!-- Update User
        <a href="/update-book">
            <button>Update Book</button>
        </a> -->

        <!-- Delete User -->
        <a href="/delete-book">
            <button>Delete Book</button>
        </a>
    </div>
</body>
```

delete.ejs × | home.ejs | update.ejs | create.ejs | # styl

views > delete.ejs > ...

```html
<!-- Add your code here -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Delete Book</title>
  <link rel="stylesheet" href="/css/styles.css">
</head>
<body>
  <h1>Delete the Book </h1>

  <form action="/delete-book" method="POST">
    <button type="submit">Delete book</button>
    <a href="/"><button type="button">Cancel</button></a>
  </form>
</body>
</html>
```

delete.ejs | home.ejs | update.ejs × | create.ejs | # styles.css | JS index.js

views > update.ejs > ...

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Update Book</title>
  <link rel="stylesheet" href="/css/styles.css">
</head>
<body>
  <h1>Update Book </h1>
  <h3>Update Book ID = <%= book.BookID %></h3>

  <form action="/update-book/<%= book.BookID %>" method="POST">
    <label>Title</label>
    <input type="text" name="Title" value="<%= book.Title %>" required>

    <label>Author</label>
    <input type="text" name="Author" value="<%= book.Author %>" required>

    <button type="submit">Update</button>
    <a href="/"><button type="button">Cancel</button></a>
  </form>
</body>
</html>
```

delete.ejs | home.ejs | update.ejs | create.ejs | styles.css | index.js

views > create.ejs > ...

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Add Book</title>
  <link rel="stylesheet" href="/css/styles.css">
</head>
<body>
  <h1>Add a New Book</h1>
  <form action="/add-book" method="POST" class="form">
    <label>Title</label>
    <input type="text" name="Title" placeholder="Book Title" required>

    <label>Author</label>
    <input type="text" name="Author" placeholder="Author Name" required>

    <button type="submit">Add Book</button>
    <a href="/"><button type="button">Back</button></a>
  </form>
</body>
</html>
```

delete.ejs | home.ejs | update.ejs | create.ejs | styles.css | index.js

public > css > styles.css > body

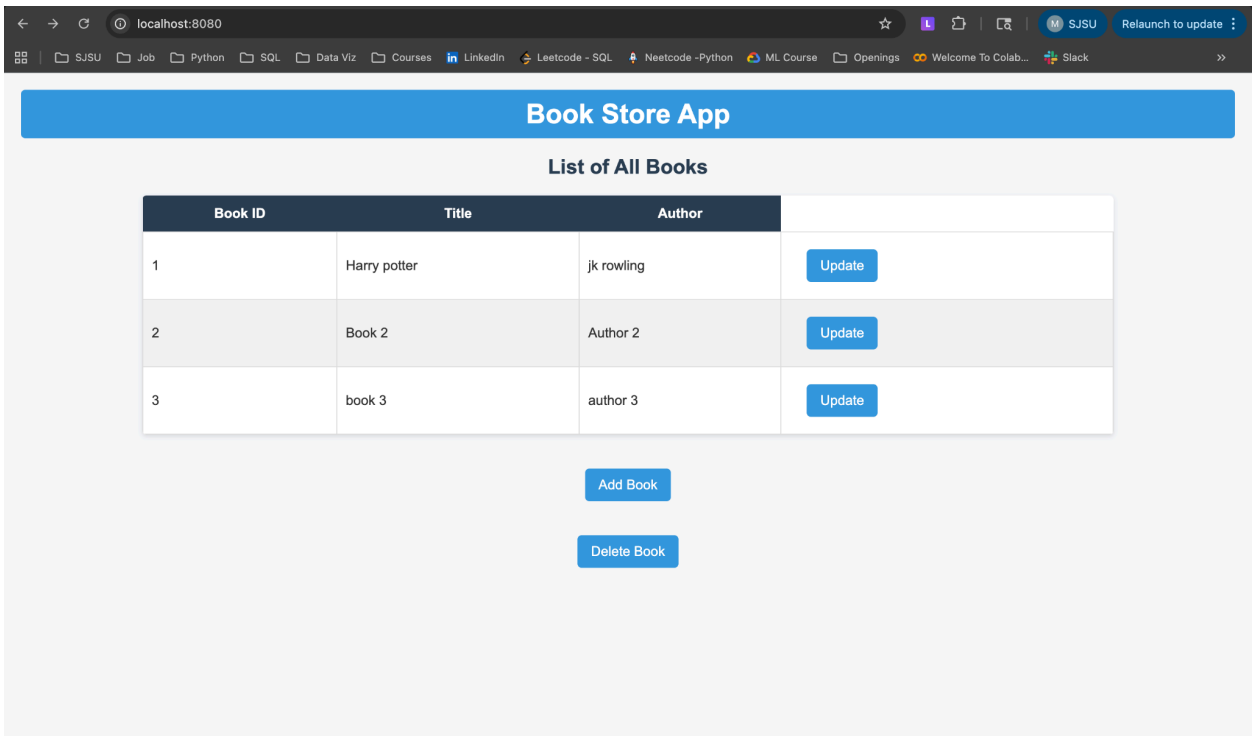
```css
body {
    font-family: Arial, sans-serif;
    margin: 0;
    padding: 20px;
    background-color: #f4f6f9;
    color: #333;
}

h1, h2 {
    color: #2c3e50;
    text-align: center;
}

button {
    margin: 10px;
    padding: 10px 16px;
    font-size: 16px;
    border: none;
    border-radius: 5px;
    background-color: #3498db;
    color: white;
    cursor: pointer;
    transition: background-color 0.3s ease;
}

button:hover {
    background-color: #2980b9;
}

a {
    margin: 10px;
    text-decoration: none;
}

.heading {
    padding: 10px;
    margin-bottom: 20px;
    text-align: center;
    background-color: #3498db;
    color: white;
    border-radius: 5px;
}

.actions-container {
    margin: 20px auto;
    display: flex;
    flex-direction: column;
    max-width: 400px;
    align-items: center;
}
```

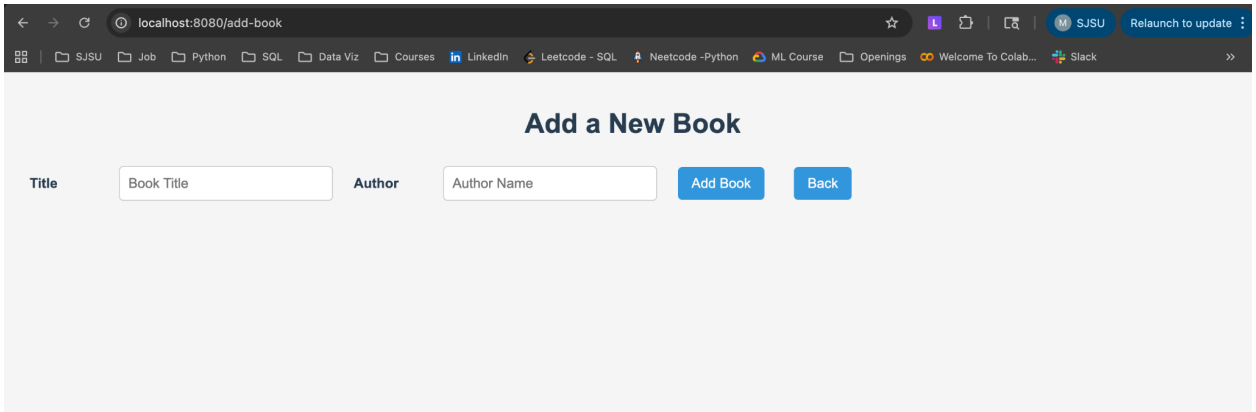
delete.ejs | home.ejs | update.ejs | create.ejs | styles.css | index.js

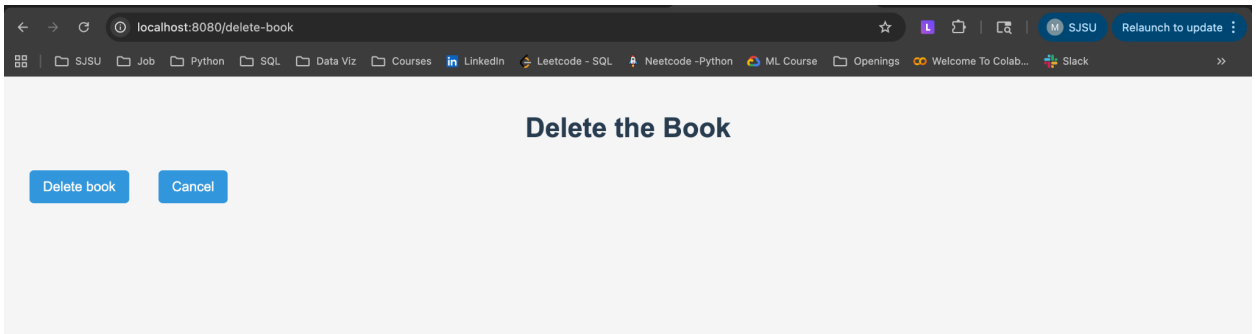
public > css > styles.css > .books-table tr:hover

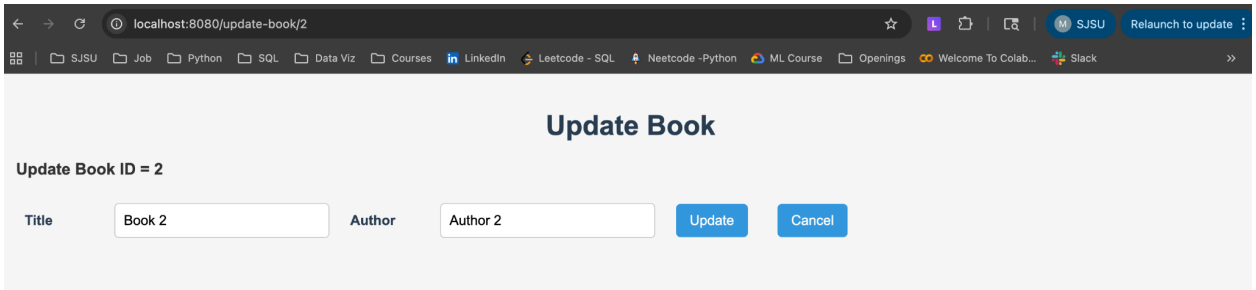
```
51
52  .books-table {
53      width: 80%;
54      margin: 20px auto;
55      border-collapse: collapse;
56      text-align: left;
57      background-color: white;
58      box-shadow: 0 2px 5px rgba(0,0,0,0.1);
59      border-radius: 5px;
60      overflow: hidden;
61  }
62
63  .books-table th {
64      background-color: #2c3e50;
65      color: white;
66      padding: 12px;
67      text-align: center;
68  }
69
70  .books-table td {
71      border: 1px solid #ddd;
72      padding: 10px;
73  }
74
75  .books-table tr:nth-child(even) {
76      background-color: #f2f2f2;
77  }
78
79  .books-table tr:hover {
80      background-color: #eaf2f8;
81  }
82
83  input[type="text"] {
84      padding: 10px;
85      margin: 10px;
86      font-size: 16px;
87      border: 1px solid #ccc;
88      border-radius: 5px;
89      width: 250px;
90      box-sizing: border-box;
91      transition: border-color 0.3s ease, box-shadow 0.3s ease;
92  }
93
94
95  input[type="text"]:focus {
96      border-color: #3498db;
97      outline: none;
98      box-shadow: 0 0 5px rgba(52, 152, 219, 0.5);
99  }
100
101
```

```
100
101
102  label {
103      font-weight: bold;
104      margin: 10px;
105      display: inline-block;
106      min-width: 80px;
107      color: #2c3e50;
108  }
109
```









Part 2. HTTP, Express, NodeJS

1. Write the code to add a new book. The user should be able to enter the Book Title and Author Name. Once the user submits the required data, the book should be added, and the user should be redirected to the home view showing the updated list of books. (2 points)

```
// add code to render the create view
app.get('/add-book', function (req, res) {
  res.render('create');
});

// add your code to create new book
app.post('/add-book', function (req, res) {
  const { Title, Author } = req.body;

  const nextId = books.length
    ? (Math.max(...books.map(b => parseInt(b.BookID, 10))) + 1).toString()
    : "1";

  books.push({ BookID: nextId, Title, Author });
  res.redirect('/');
});
```

2. Write the code to update the book with ID 1 to title:” Harry Potter”, Author Name: “J.K Rowling”. After submitting the data, redirect to the home view and show the updated data in the list of books. (2 points)

```
app.get('/update-book/:id', function (req, res) {
  const bookId = req.params.id;
  const book = books.find(b => b.BookID === bookId);

  if (!book) {
    return res.status(404).send("Book not found");
  }

  res.render('update', { book });
});


app.post('/update-book/:id', function (req, res) {
  const bookId = req.params.id;
  const { Title, Author } = req.body;

  const book = books.find(b => b.BookID === bookId);
  if (book) {
    book.Title = Title;
    book.Author = Author;
  }

  res.redirect('/');
});
```

3. Write the code to delete the book with the highest ID. After submitting the data, redirect to the home view and show the updated data in the list of books. (2 points)

```javascript
// add code to render the delete view
app.get('/delete-book', function (req, res) {
  res.render('delete');
});

// add code to delete a book
app.post('/delete-book', function (req, res) {
  if (!books.length) return res.redirect('/');

  const maxId = Math.max(...books.map(b => parseInt(b.BookID, 10)));
  const idx = books.findIndex(b => parseInt(b.BookID, 10) === maxId);
  if (idx > -1) books.splice(idx, 1);

  res.redirect('/');
});
```

PART 3. Stateful Agent Graph

Step 2: Setting up the State

First, define the shared AgentState using Python's TypedDict. This class will represent the memory of our system. It needs to hold the initial inputs, the outputs from each agent, and a turn counter to prevent infinite loops.

```python
class AgentState(TypedDict):
    title: str
    content: str
    email: str
    strict: bool
    task: str
    llm: Any
    planner_proposal: Dict[str, Any]
    reviewer_feedback: Dict[str, Any]
    turn_count: int
```

Step 3: Creating the Agent Nodes

Next, convert your Planner and Reviewer logic into standalone functions. Each function must:

Accept state: AgentState as its only argument.

Perform its task (e.g., call the LLM).

Return a dictionary containing only the keys of the AgentState it wants to update.

agents_demo.py > ...

```python

def planner_node(state: AgentState) -> Dict[str, Any]:
    print("---NODE: Planner ---")
    llm = state["llm"]

    user_prompt = f"TITLE:\n{state['title']}\n\nCONTENT:\n{state['content']}"
    response = llm.invoke([SystemMessage(PLANNER_SYS), HumanMessage(user_prompt)])
    proposal = parse_json_maybe(response.content)
    proposal = ensure_message(proposal)

    return {"planner_proposal": proposal, "reviewer_feedback": {} }


def reviewer_node(state: AgentState) -> Dict[str, Any]:
    print("---NODE: Reviewer ---")
    llm = state["llm"]

    planner_out = state.get("planner_proposal", {})
    user_prompt = json.dumps({
        "title": state["title"],
        "content": state["content"],
        "planner": planner_out
    }, ensure_ascii=False)

    response = llm.invoke([SystemMessage(REVIEWER_SYS), HumanMessage(user_prompt)])
    feedback = parse_json_maybe(response.content)
    feedback = ensure_message(feedback)

    return {"reviewer_feedback": feedback}

```

Step 4: Building the Supervisor (The Router)

The supervisor is the "brain" of the operation. It doesn't do the work; it directs it. We split its logic into two parts:

1. **A State-Updating Node (supervisor_node):** This node's only job is to modify the state, like incrementing the turn counter.
2. **A Routing Function (router_logic):** This function *reads* the state and decides where to go next by returning a string.

agents_demo.py

agents_demo.py > router_logic

```python
    return {"reviewer_feedback": feedback}


def supervisor_node(state: AgentState) -> Dict[str, Any]:
    """Update turn counter each round."""
    return {"turn_count": state["turn_count"] + 1}


def router_logic(state: AgentState) -> str:
    """Decide next node based on state."""
    if state["turn_count"] > 5:
        return END

    if not state.get("planner_proposal"):
        return "planner"

    if not state.get("reviewer_feedback"):
        return "reviewer"

    if state["reviewer_feedback"].get("issues"):
        return "planner"


    return END
```

Step 5: Assembling the Graph

Now, wire everything together in your main function.

```python
def build_graph():
    workflow = StateGraph(AgentState)

    workflow.add_node("planner", planner_node)
    workflow.add_node("reviewer", reviewer_node)
    workflow.add_node("supervisor", supervisor_node)

    workflow.add_conditional_edges("supervisor", router_logic, {
        "planner": "planner",
        "reviewer": "reviewer",
        END: END,
    })

    workflow.add_edge("planner", "supervisor")
    workflow.add_edge("reviewer", "supervisor")

    workflow.set_entry_point("supervisor")
    return workflow.compile()
```

```python
if __name__ == "__main__":
    graph = build_graph()

    initial_state: AgentState = {
        "title": "How Reading Before Bed Improves Sleep",
        "content": """Reading before bed helps the brain relax and improves sleep
    quality by reducing stress and calming mental activity. Unlike screens,
    books do not emit blue light, making them a healthier option for winding
    down.

    Incorporating reading into a nightly routine signals the body that it's time to sleep,
    helping people fall asleep faster and wake up more refreshed.

    Research shows that even six minutes of reading can lower stress levels by more than 60%,
    making it one of the most effective bedtime habits.""",
        "email": "mounashree.prasanna@sjsu.edu",
        "strict": True,
        "task": "summarize",
        "llm": ChatOllama(model="phi3:mini", temperature=0.2),
        "planner_proposal": {},
        "reviewer_feedback": {},
        "turn_count": 0,
    }

    print("=== Running Graph ===")
    for event in graph.stream(initial_state):
        print("Event:", event)
    print("=== Done ===")
```

Step 6: Running and Testing

Finally, invoke your graph with the initial state and use the .stream() method to see the output from each step.

To test your correction loop, temporarily modify your reviewer_node to always return an issue, and watch the graph route the task back to the planner.

Reviewer with no issues:

```
TERMINAL
(tinyagents) mounashreeprasanna@Mounashrees-MacBook-Air langgraph-agents % python agents_demo.py

=== Running Graph ===
Event: {'supervisor': {'turn_count': 1}}
---NODE: Planner ---
Event: {'planner': {'planner_proposal': {'thought': 'The content discusses how pre-sleep activities, specifically reading, affect sleep quality and
 mental health.', 'message': 'Reading before bed improves relaxation and reduces stress for better sleep', 'data': {'tags': ['reading', 'bedtime ro
utine', 'blue light']}, 'summary': 'Research shows that even six minutes of pre-sleep reading can lower stress levels by more than 60%, making it o
ne of the most effective bedtime habits.', 'issues': []}, 'reviewer_feedback': {}}}
Event: {'supervisor': {'turn_count': 2}}
---NODE: Reviewer ---
Event: {'reviewer': {'reviewer_feedback': {'thought': 'The content discusses how pre-sleep activities, specifically reading, affect sleep quality a
nd mental health.', 'message': 'Reading before bed improves relaxation and reduces stress for better sleep', 'data': {'tags': ['reading', 'bedtime
routine', 'blue light']}, 'summary': 'Research shows that even six minutes of pre-sleep reading can lower stress levels by more than 60%, making it
 one of the most effective bedtime habits.', 'issues': []}}}
Event: {'supervisor': {'turn_count': 3}}
=== Done ===
```

Reviewer with issues:

```
TERMINAL
(tinyagents) mounashreeprasanna@Mounashrees-MacBook-Air langgraph-agents % python agents_demo.py

=== Running Graph ===
Event: {'supervisor': {'turn_count': 1}}
---NODE: Planner ---
Event: {'planner': {'planner_proposal': {'thought': 'The content discusses how pre-sleep activities, specifically reading without screens and blue
light emissions, improve sleep quality.', 'message': 'Reading before bed improves sleep by reducing stress levels', 'data': {'tags': ['reading', 'b
edtime routine', 'blue light']}, 'summary': 'Research shows that even six minutes of pre-sleep book reading can lower stress levels significantly.'
}, 'reviewer_feedback': {}}}
Event: {'supervisor': {'turn_count': 2}}
---NODE: Reviewer ---
Event: {'reviewer': {'reviewer_feedback': {'thought': 'The content discusses how pre-sleep activities, specifically screen time and blue light expo
sure before bed, negatively impact sleep quality.', 'message': 'Avoiding screens an hour before bed improves sleep by reducing mental stimulation',
 'data': {'tags': ['screen time', 'blue light']}, 'summary': 'Research shows that avoiding screens for at least one hour before bed can improve the
 ability to fall asleep and wake up refreshed.', 'issues': ['too vague']}}}
Event: {'supervisor': {'turn_count': 3}}
---NODE: Planner ---
Event: {'planner': {'planner_proposal': {'thought': 'The content discusses how pre-sleep activities, specifically reading, contribute to better sle
ep quality and reduced stress.', 'message': '<Reading before bed helps relaxation><Healthy alternative to screens><Improves nightly routine for fas
ter sleep onset>', 'data': {'tags': ['reading', 'bedtime habits', 'stress reduction']}, 'summary': 'The article explains how reading pre-sleep aids
 in stress relief and promotes better, quicker sleep.'}, 'reviewer_feedback': {}}}
Event: {'supervisor': {'turn_count': 4}}
---NODE: Reviewer ---
Event: {'reviewer': {'reviewer_feedback': {'thought': 'The content discusses the benefits of bedtime reading for relaxation and improved sleep qual
ity.', 'message': '<Reading before bed helps with relaxation><Healthy alternative to screens><Improves nightly routine>', 'data': {'tags': ['readin
g', 'bedtime habits', 'stress reduction']}, 'summary': 'The article explains how reading pre-sleep aids in stress relief and promotes better, quick
er sleep.', 'issues': []}}}
Event: {'supervisor': {'turn_count': 5}}
=== Done ===
```